Your MySQL assessment workbook

A detailed refresher for someone who has already taken a data science course.

How to use this before Sunday

Read actively: cover an example, write the query yourself, then explain why each clause belongs there. Use the note lines to record your own mistakes and alternative solutions.

Your assessment screenshot lists two SQL (Intermediate) questions and one SQL (Intermediate) - Database item within a 75-minute test. It does not reveal the exact topics or question formats. This guide targets broadly useful intermediate SQL, not a claimed ZS question bank.

Suggested route

First pass: pages 3-11 for retrieval, joins, aggregation and decomposition. **Second pass:** pages 13-18 for windows, dates and reusable patterns. **Database review:** pages 19-22. **Final check:** pages 23-26.

Scope and conventions

This workbook focuses entirely on **MySQL 8+**, your chosen assessment language. Use MySQL for every practice problem. CTEs and window functions require MySQL 8.0 or later; a few newer features have explicit version notes.

This is an assessment-focused reference, not an exhaustive SQL language manual. Stored procedures, triggers, recursive hierarchy queries and engine administration are lower priority for this short preparation window.

> **Most important habit:** Before writing SQL, finish this sentence: "Each output row represents one _____." That is the result grain.

Priority: joins and row counts → grouping and rates → subqueries and CTEs → windows and dates → database fundamentals.

MY NOTES / QUESTIONS TO REVISIT

Map of the workbook

Use these page numbers to jump directly to a weak area.

03 SELECT and logical processing order
04 Filtering, sorting and distinct rows
05 NULL and three-valued logic
06 Joins and self-joins
07 Join multiplication and missing matches
08 Aggregates, GROUP BY and HAVING
09 CASE, rates and weighted averages
10 Subqueries, EXISTS and NOT EXISTS
11 CTEs and multi-step solutions
12 Set operations and duplicate semantics
13 Window functions and ranking
14 LAG, LEAD and window frames
15 Dates, timestamps and calendar logic
16 Strings, numeric functions and types
17 Reusable interview patterns
18 Worked business query
19 Tables, constraints and schema design
20 Data changes and transactions
21 Indexes, query plans and normalization
22 MySQL isolation and syntax essentials
23 Debugging and assessment checklist
24 Practice prompts
25 Practice solutions
26 One-page syntax reference
27 Study plan, references and notes

Shared example schema

```
customers(customer_id PK, name, region)
orders(order_id PK, customer_id FK, order_ts,
       amount DECIMAL(12,2), status)
employees(employee_id PK, name, department_id,
          manager_id, salary)
order_items(order_id, product_id, quantity, unit_price)
events(event_id PK, user_id, event_ts, event_type)
```

Assume IDs are non-null unique keys where marked PK; orders.customer_id references customers.customer_id. Unless a prompt says otherwise, orders.amount is one order total, timestamps are stored consistently, and multiple orders per customer are allowed. Do not infer that every customer has an order.

MY NOTES / QUESTIONS TO REVISIT

SELECT and logical processing order

Write clauses in SQL order; reason about them in logical order.

```
SELECT c.region,
       COUNT(*) AS order_count,
       SUM(o.amount) AS revenue
FROM customers AS c
JOIN orders AS o
  ON o.customer_id = c.customer_id
WHERE o.status = 'completed'
GROUP BY c.region
HAVING SUM(o.amount) > 1000
ORDER BY revenue DESC, c.region ASC
LIMIT 5;
```

What each clause does

FROM / JOIN / ON: choose sources and matching conditions. **WHERE:** filter individual rows before aggregation. **GROUP BY:** form groups. **HAVING:** filter groups after aggregation. **SELECT:** choose expressions and output names. **ORDER BY:** sort the result. **LIMIT:** take a slice of that sorted result.

A useful conceptual sequence is FROM/JOIN → WHERE → GROUP BY → HAVING → window evaluation → SELECT → DISTINCT → ORDER BY → LIMIT. The optimizer may physically execute a different plan while preserving semantics.

> **Alias trap:** A SELECT alias generally cannot be referenced in WHERE or another expression in the same SELECT list. Put the calculation in a CTE if you need to reuse it. ORDER BY can use a SELECT alias; MySQL also permits aliases in HAVING.

Read the example

One output row is one region. Only completed orders contribute. Regions must exceed 1,000 in total revenue. The final output contains at most five regions, sorted by revenue with a region tie-breaker.

SQL normally preserves duplicates. SELECT does not imply DISTINCT. Use single quotes for string literals and qualify ambiguous column names with table aliases. End statements with a semicolon.

MY NOTES / QUESTIONS TO REVISIT

Filtering, sorting and distinct rows

Know the operators well enough to translate plain English without guessing.

```
SELECT DISTINCT customer_id
FROM orders
WHERE status IN ('completed', 'shipped')
  AND amount BETWEEN 50 AND 200
  AND order_ts >= '2026-10-01'
  AND order_ts <  '2026-11-01'
ORDER BY customer_id;
```

Comparison: =, <> (not equal), >, >=, <, <=. **Boolean:** AND, OR, NOT. **Membership:** IN (...). **Range:** BETWEEN includes both endpoints. **Missing:** IS NULL / IS NOT NULL. **Pattern:** LIKE.

Parentheses prevent logic bugs

```
WHERE (region = 'East' OR region = 'West')
  AND amount >= 100
-- AND binds more tightly than OR.
-- Without parentheses, East rows could bypass amount.
```

LIKE and ordering

LIKE 'A%': starts with A. **LIKE '%son':** ends with son. **LIKE '_a%':** second character is a. Percent matches zero or more characters; underscore matches exactly one. Case sensitivity depends on MySQL collation; use an explicit ESCAPE clause when matching literal wildcard characters.

ORDER BY amount DESC, order_id ASC makes ties deterministic if order_id is unique. No ORDER BY means no guaranteed output order. LIMIT without ordering is not a reliable top-N query.

DISTINCT applies to the complete selected tuple. SELECT DISTINCT customer_id, status returns unique customer/status pairs, not one row per customer. LIMIT 10 OFFSET 20 skips twenty rows and returns up to ten; use a deterministic sort.

> Dates and timestamps: prefer a half-open interval [start, next_start). An inclusive month-end date often accidentally excludes most of the last day.

MY NOTES / QUESTIONS TO REVISIT

NULL and three-valued logic

NULL means missing or unknown; it is not zero, an empty string or a normal value.

```
SELECT customer_id,
       COALESCE(region, 'Unknown') AS region_label
FROM customers
WHERE region IS NULL;

SELECT 100.0 * SUM(amount) / NULLIF(COUNT(*), 0)
FROM orders;
```

Truth rules that appear in hidden tests

Comparisons such as NULL = 5, NULL <> 5 and NULL = NULL evaluate to UNKNOWN. WHERE and HAVING retain only TRUE rows, so WHERE region <> 'East' excludes NULL regions too. To include them, write region <> 'East' OR region IS NULL.

COALESCE(a,b,c) returns the first non-null argument. **NULLIF(a,b)** returns NULL when a = b; otherwise it returns a. Dividing by NULL yields NULL, which is useful for an undefined rate.

Aggregate behavior

COUNT(*) counts rows; COUNT(column) counts non-null values. COUNT(DISTINCT column) counts distinct non-null values. SUM, AVG, MIN and MAX ignore null inputs. Without GROUP BY, an empty input gives COUNT = 0 and SUM/AVG/MIN/MAX = NULL; with GROUP BY, no groups means no result rows.

> **Do not automatically replace NULL with 0.** AVG(COALESCE(score,0)) treats missing scores as actual zeros. COALESCE(AVG(score),0) only replaces a missing final average. These answer different questions.

NOT IN can surprise you

If a NOT IN subquery contains NULL, a nonmatching candidate can evaluate to UNKNOWN rather than TRUE. Prefer NOT EXISTS for missing-related-row questions, or explicitly eliminate NULL from the subquery. A key declared NOT NULL also removes this particular risk.

Mini-check: for values [10, NULL, 20], COUNT(*) = 3, COUNT(value) = 2, SUM(value) = 30 and AVG(value) = 15.

MY NOTES / QUESTIONS TO REVISIT

Joins and self-joins

A join builds matching row combinations. Choose which unmatched rows should survive.

```
SELECT c.customer_id, c.name, o.order_id
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id;

SELECT e.name AS employee, m.name AS manager
FROM employees AS e
LEFT JOIN employees AS m
  ON e.manager_id = m.employee_id;
```

INNER JOIN: only matching pairs. **LEFT JOIN:** all left rows, with NULL right columns when unmatched. **RIGHT JOIN:** mirror of LEFT JOIN; swapping table order often makes it easier to read. **FULL OUTER JOIN:** matched rows and unmatched rows from both sides; not native in MySQL. **CROSS JOIN:** all combinations.

ON versus WHERE in an outer join

```
-- Keep every customer; attach only completed orders.
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id
 AND o.status = 'completed'

-- A WHERE o.status = 'completed' afterward would
-- remove unmatched rows and change the result.
```

ON defines which pairs match. WHERE then filters the joined rows. A predicate that rejects NULL on the right side of a LEFT JOIN can remove the very unmatched rows you meant to preserve.

Relationships and keys

One-to-one: at most one matching row per side. One-to-many: a customer repeats for each order. Many-to-many: often modeled by a bridge table and can produce many combinations. A self-join is an ordinary join where aliases represent different roles in the same table.

> Always state the join key and expected cardinality. Matching on customer name rather than a unique customer ID can create false matches.

MY NOTES / QUESTIONS TO REVISIT

Join multiplication and missing matches

Many SQL errors are correct syntax applied at the wrong grain.

A tiny example

Customer 1 has order A worth 100 and order B worth 50. A has three items; B has two. Joining orders to order_items produces five rows. SUM(o.amount) becomes 100 + 100 + 100 + 50 + 50 = 400, even though actual revenue is 150.

> **DISTINCT is not a general repair.** SUM(DISTINCT amount) would also collapse different orders with the same amount. Fix the grain or choose the correct metric.

```
-- Aggregate item facts before joining to order facts.
WITH item_totals AS (
  SELECT order_id, SUM(quantity) AS units
  FROM order_items
  GROUP BY order_id
)
SELECT o.customer_id,
       SUM(o.amount) AS revenue,
       SUM(COALESCE(i.units, 0)) AS units
FROM orders AS o
LEFT JOIN item_totals AS i ON i.order_id = o.order_id
GROUP BY o.customer_id;
```

Count matches without counting placeholders

```
SELECT c.customer_id, COUNT(o.order_id) AS n_orders
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
GROUP BY c.customer_id;
```

COUNT(*) would give an unmatched customer a count of 1 because the LEFT JOIN still produces a placeholder row. COUNT(o.order_id) returns 0 because order_id is NULL on that placeholder.

Customers with no orders

```
SELECT c.customer_id
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Check a non-nullable right-side key for absence. A nullable business column could also be NULL on a genuine matched row.

MY NOTES / QUESTIONS TO REVISIT

Aggregates, GROUP BY and HAVING

Aggregation changes the grain: many input rows become one output row per group.

```
SELECT customer_id,
       COUNT(*) AS n_orders,
       SUM(amount) AS total_spend,
       ROUND(AVG(amount), 2) AS avg_order,
       MIN(order_ts) AS first_order,
       MAX(order_ts) AS latest_order
FROM orders
WHERE status = 'completed'
GROUP BY customer_id
HAVING COUNT(*) >= 2
ORDER BY total_spend DESC, customer_id;
```

Rules worth memorizing

Use WHERE for row conditions such as status = 'completed'. Use HAVING for group conditions such as COUNT(*) >= 2. With MySQL ONLY_FULL_GROUP_BY enabled, ungrouped selected columns must be functionally dependent on the grouping keys. As a simple habit, group every selected nonaggregated expression unless that dependency is clear.

GROUP BY customer_id, status creates one row per customer/status combination. GROUP BY does not promise sorted output. Use COUNT(DISTINCT customer_id) to count customers rather than orders.

Average of averages is often wrong

Group A has one order worth 100. Group B has nine orders worth 10 each. The average of group averages is (100 + 10)/2 = 55; the average order value is (100 + 90)/10 = 19. Combine sums and counts when groups have unequal sizes.

```
WITH customer_stats AS (
  SELECT customer_id, SUM(amount) AS spend,
         COUNT(*) AS n
  FROM orders
  GROUP BY customer_id
)
SELECT SUM(spend) / NULLIF(SUM(n), 0) AS overall_avg
FROM customer_stats;
```

> Name the denominator before calculating a metric: all rows, non-null observations, unique customers, completed orders, or eligible users? Most rate mistakes start here.

MY NOTES / QUESTIONS TO REVISIT

CASE, rates and weighted averages

Conditional aggregation handles several business metrics in a single grouped query.

```
SELECT customer_id,
       COUNT(*) AS total_orders,
       SUM(CASE WHEN status = 'completed'
                THEN 1 ELSE 0 END) AS completed_orders,
       SUM(CASE WHEN status = 'completed'
                THEN amount ELSE 0 END) AS revenue,
       ROUND(100.0 * SUM(CASE WHEN status = 'completed'
                              THEN 1 ELSE 0 END)
             / NULLIF(COUNT(*), 0), 2) AS completed_pct
FROM orders
GROUP BY customer_id;
```

Searched CASE: CASE WHEN condition THEN value WHEN ... ELSE value END. Conditions are tested in order; the first true branch wins. Without ELSE, unmatched cases become NULL. CASE can appear in SELECT, ORDER BY and aggregate arguments.

> **Counting trap:** COUNT(CASE WHEN condition THEN 1 ELSE 0 END) counts both 1 and 0. Use SUM(... THEN 1 ELSE 0 END), or COUNT(CASE WHEN condition THEN 1 END).

Weighted average

```
SELECT product_id,
       ROUND(SUM(quantity * unit_price)
             / NULLIF(SUM(quantity), 0), 2) AS avg_price
FROM order_items
GROUP BY product_id;
```

The denominator is total units, not number of item rows. If NULL prices or quantities are possible, define whether such rows are excluded from both numerator and denominator. Do not count units with unknown prices unless that is intended.

Zero denominators and numeric types

NULLIF(denominator,0) avoids a zero denominator. Return NULL when a rate is undefined, or COALESCE the final result to 0 if the question requires zero. Use 100.0 or a decimal cast for portable noninteger division; round the final result, not every intermediate calculation.

MY NOTES / QUESTIONS TO REVISIT

Subqueries, EXISTS and NOT EXISTS

Use a subquery when one question depends on the result of another.

```
-- Scalar subquery: one value.
SELECT employee_id, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Correlated subquery: depends on the outer row.
SELECT e.employee_id, e.salary
FROM employees AS e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees AS e2
  WHERE e2.department_id = e.department_id
);
```

A scalar subquery must return at most one row and one column; an aggregate without GROUP BY supplies one row. A multirow result used as a scalar raises an error. A correlated subquery references an outer alias. The optimizer may transform it; do not assume a specific execution strategy.

Existence rather than row duplication

```
SELECT c.customer_id
FROM customers AS c
WHERE EXISTS (
  SELECT 1 FROM orders AS o
  WHERE o.customer_id = c.customer_id
    AND o.status = 'completed'
);

SELECT c.customer_id
FROM customers AS c
WHERE NOT EXISTS (
  SELECT 1 FROM orders AS o
  WHERE o.customer_id = c.customer_id
);
```

EXISTS is true when at least one row is found. The selected value (often 1) does not matter. NOT EXISTS is true when no row is found. These patterns preserve one row per qualifying outer row without multiplying it by matches.

Use IN for membership in a list or a single-column subquery. For exclusion, review the NOT IN / NULL behavior on page 5. A derived table is a subquery in FROM and should have an alias: FROM (SELECT ...) AS x.

MY NOTES / QUESTIONS TO REVISIT

CTEs and multi-step solutions

WITH gives names to intermediate results so each stage has one clear job.

```
WITH completed AS (
  SELECT customer_id, amount
  FROM orders
  WHERE status = 'completed'
), customer_spend AS (
  SELECT customer_id, SUM(amount) AS spend
  FROM completed
  GROUP BY customer_id
), threshold AS (
  SELECT AVG(spend) AS avg_spend
  FROM customer_spend
)
SELECT s.customer_id, s.spend
FROM customer_spend AS s
CROSS JOIN threshold AS t
WHERE s.spend > t.avg_spend
ORDER BY s.spend DESC, s.customer_id;
```

Explain the decomposition

completed: eligible orders. **customer_spend:** one row per customer with completed purchases. **threshold:** one row holding the average of those customer totals. **Final:** customers whose total exceeds that customer-level average.

This compares customer totals with other customer totals. Comparing each total to AVG(order.amount) would compare different grains. Customers without completed orders are excluded here; if the definition includes them with zero spend, build customer_spend from customers using a LEFT JOIN.

Syntax and scope

Write one WITH followed by comma-separated definitions. Later CTEs can reference earlier ones. A CTE exists for the immediately following statement; it is not a permanent table. Do not add a semicolon between the CTE and its final SELECT.

> CTEs are a reasoning tool, not a guaranteed performance improvement. MySQL may merge or materialize them. Use them to separate filtering, aggregation, ranking and final formatting.

Recursive CTEs use an anchor plus a recursive step, often joined by UNION ALL. They are useful for hierarchies and date series, but are lower priority than ordinary CTEs for this review.

MY NOTES / QUESTIONS TO REVISIT

Set operations and duplicates

Joins combine columns through relationships. Set operations combine compatible result rows.

```
SELECT customer_id FROM online_customers
UNION
SELECT customer_id FROM store_customers;

SELECT customer_id FROM online_customers
UNION ALL
SELECT customer_id FROM store_customers;
```

UNION: stacks results and removes duplicate complete rows. **UNION ALL:** stacks results and keeps duplicates. **INTERSECT:** rows present in both results. **EXCEPT:** rows in the first result that are absent from the second. MySQL supports INTERSECT and EXCEPT starting with 8.0.31. If the assessment uses an earlier 8.0 release, use EXISTS / NOT EXISTS instead.

Shape requirements

Each input needs the same number of columns in corresponding order and compatible types. Column names normally come from the first SELECT. Put the final ORDER BY after the combined query. Parenthesize inputs with their own sorting/limiting using MySQL parentheses syntax.

Use UNION ALL when duplicates are meaningful

Combining January and February transaction rows usually calls for UNION ALL. UNION could silently remove two genuinely distinct transactions if their selected columns are identical. Include identifiers when they are part of the intended row meaning.

MySQL FULL OUTER JOIN workaround

```
SELECT a.id AS a_id, b.id AS b_id
FROM a LEFT JOIN b ON a.id = b.id
UNION ALL
SELECT a.id AS a_id, b.id AS b_id
FROM b LEFT JOIN a ON a.id = b.id
WHERE a.id IS NULL;
```

Assume a.id is non-null on real rows. The first branch includes all matches and unmatched a rows; the second adds only unmatched b rows. UNION ALL avoids an extra deduplication that could alter legitimate duplicates.

> Do not confuse SELECT DISTINCT with COUNT(DISTINCT x): the former deduplicates output rows; the latter counts unique non-null values inside each group.

MY NOTES / QUESTIONS TO REVISIT

Window functions and ranking

Windows calculate across related rows while preserving the rows being analyzed.

```
WITH ranked AS (
  SELECT employee_id, department_id, salary,
         DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC
         ) AS salary_rank
  FROM employees
  WHERE salary IS NOT NULL
)
SELECT employee_id, department_id, salary
FROM ranked
WHERE salary_rank <= 3;
```

OVER: makes the function a window calculation. **PARTITION BY:** resets the calculation per group; omit it for one partition. **ORDER BY inside OVER:** defines calculation order. It does not guarantee final output order; add an outer ORDER BY if required.

The tie distinction

```
Salary:        100  100   90   80
ROW_NUMBER:      1    2    3    4
RANK:            1    1    3    4
DENSE_RANK:      1    1    2    3
```

ROW_NUMBER: exactly one sequence number per row; add a unique tie-breaker when selecting a specific row. **RANK:** ties share rank and leave gaps. **DENSE_RANK:** ties share rank without gaps. Top three distinct salaries calls for DENSE_RANK; exactly three employees calls for ROW_NUMBER.

> Do not add employee_id to DENSE_RANK's ORDER BY when equal salaries should tie. The rank uses the complete ordering tuple, so a unique ID would break the ties.

MySQL cannot filter a window result directly in WHERE. Compute it in a CTE or derived table, then filter in the outer query. MySQL does not support QUALIFY.

Other useful functions: SUM(x) OVER (...), AVG(x) OVER (...), COUNT(*) OVER (...), LAG, LEAD, FIRST_VALUE and LAST_VALUE.

MY NOTES / QUESTIONS TO REVISIT

LAG, LEAD and window frames

Distinguish previous rows from previous calendar days.

```
WITH daily AS (
  SELECT DATE(order_ts) AS day, SUM(amount) AS revenue
  FROM orders
  WHERE status = 'completed'
  GROUP BY DATE(order_ts)
)
SELECT day, revenue,
       LAG(revenue) OVER (ORDER BY day) AS prev_revenue,
       SUM(revenue) OVER (
         ORDER BY day
         ROWS BETWEEN UNBOUNDED PRECEDING
                  AND CURRENT ROW
       ) AS running_revenue,
       AVG(revenue) OVER (
         ORDER BY day
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS avg_last_7_rows
FROM daily
ORDER BY day;
```

Offsets and differences

LAG(x) reads the preceding row; LEAD(x) reads the following row. LAG(x,2,0) reads two rows back and returns 0 if that row does not exist. It does not replace a NULL value found in an existing row. To filter by a change, calculate the lag in one CTE and compare in the next query.

ROWS versus RANGE

ROWS counts physical rows in the ordered partition. **RANGE** uses ordering-value boundaries and can include all peers with the same ordering value. Ordered aggregate windows often default to a peer-sensitive RANGE frame. Specify the intended frame explicitly for a running calculation.

> Seven rows are seven days only if there is exactly one row for every calendar day. Missing dates require a calendar table/date series and zero filling when the metric includes no-sale days. The first six rows also have partial frames.

LAST_VALUE often surprises people because an ordered default frame ends at the current peer group. For the last value of an entire partition, use ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING, with deterministic ordering.

MY NOTES / QUESTIONS TO REVISIT

Dates, timestamps and calendar logic

Use MySQL date functions and check interval boundaries carefully.

```
-- MySQL 8+
SELECT DATE(order_ts) AS order_day,
       EXTRACT(YEAR FROM order_ts) AS yr,
       DATE_FORMAT(order_ts, '%Y-%m') AS month_key
FROM orders;

-- Fixed month: includes all timestamps in October.
WHERE order_ts >= '2026-10-01'
  AND order_ts <  '2026-11-01'

-- Previous calendar date, not merely previous row.
SELECT a.day
FROM daily_weather AS a
JOIN daily_weather AS b
  ON a.day = DATE_ADD(b.day, INTERVAL 1 DAY)
WHERE a.temperature > b.temperature;
```

Useful MySQL expressions

DATE(ts): date portion. **CURRENT_DATE:** current date; NOW() gives date and time. **DATE_ADD(d, INTERVAL 7 DAY):** add seven days. **DATE_SUB(d, INTERVAL 1 MONTH):** subtract a month. **DATEDIFF(end,start):** date-part difference in days. **TIMESTAMPDIFF(MINUTE,start,end):** integer minute difference.

Define the time question precisely

A calendar day, last 24 hours, trailing seven calendar days and previous seven observed dates are different. State whether boundaries are inclusive, which time zone defines a day, and whether no-activity days contribute zeros. For the seven-day inclusive window ending on d, the start date is d minus six days.

Group by both year and month, or a year-month key. Grouping only by MONTH(order_ts) mixes October across years. Avoid formatting to a string just to perform date comparisons.

> If a timestamp column is indexed, a direct range predicate can be more index-friendly than WHERE DATE(order_ts) = ... Filtering one fixed day can use ts >= day AND ts < day + INTERVAL 1 DAY.

MY NOTES / QUESTIONS TO REVISIT

Strings, numeric functions and types

Use functions deliberately; a small type mismatch can change the whole answer.

```
SELECT customer_id,
       CONCAT(UPPER(LEFT(TRIM(name), 1)),
              LOWER(SUBSTRING(TRIM(name), 2))) AS label
FROM customers;

SELECT ROUND(123.456, 2), ABS(-7),
       CEIL(2.1), FLOOR(2.9), MOD(7, 3),
       CAST('12.50' AS DECIMAL(10,2));
```

Strings to recognize

TRIM: remove boundary spaces by default. **UPPER / LOWER:** change case. **CONCAT:** join strings. **SUBSTRING(s,start,length):** extract text (positions start at 1). **LEFT / RIGHT:** take boundary characters. **REPLACE(s,old,new):** replace matching text. **CHAR_LENGTH:** count characters; MySQL LENGTH counts bytes.

MySQL CONCAT returns NULL if any argument is NULL. Use COALESCE when missing pieces should become empty strings. Collations determine many comparison and ordering behaviors; LOWER alone is not a universal text-normalization policy.

MySQL regular expressions

```
-- MySQL: anchored, simple format test.
WHERE REGEXP_LIKE(code, '^[A-Z][0-9]{3}$', 'c')
-- ^ start; $ end; [] character set; {3} repetition.
-- 'c' requests case-sensitive matching in MySQL.
```

Types and numeric precision

INT/BIGINT store whole numbers; DECIMAL(p,s) stores fixed-precision decimal values; FLOAT/DOUBLE are approximate; VARCHAR stores variable-length text; DATE stores a date; DATETIME stores date/time as entered; TIMESTAMP is converted between the session time zone and UTC for storage/retrieval.

CAST(value AS type) makes conversion explicit. Do not rely on implicit conversion between text and numbers. MySQL / performs ordinary division; DIV performs integer division, discarding the fractional part. Use DECIMAL for exact monetary calculations and decimal operands for rates.

> Round at the output boundary. ROUND, FLOOR and CEIL are different operations. Treat percentages such as 25 and proportions such as 0.25 as different requested formats.

MY NOTES / QUESTIONS TO REVISIT

Reusable interview patterns

Translate the wording into a known structure, then check ties and missing values.

Second-highest distinct value

```
SELECT MAX(salary) AS SecondHighestSalary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

MAX over an empty set returns NULL, so this still gives one row when no second distinct salary exists. ORDER BY ... LIMIT 1 OFFSET 1 may instead return no row. Match the required output shape.

Latest row per customer: exactly one winner

```
WITH ranked AS (
  SELECT o.*, ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY order_ts DESC, order_id DESC
  ) AS rn
  FROM orders AS o
)
SELECT customer_id, order_id, order_ts, amount
FROM ranked
WHERE rn = 1;
```

The ID tie-breaker makes the result deterministic under a stated rule. If every order tied at the earliest/latest timestamp must survive, use RANK over that timestamp or join to grouped MIN/MAX values instead.

Conditional uniqueness

```
SELECT COUNT(DISTINCT CASE
         WHEN status = 'completed' THEN customer_id
       END) AS purchasing_customers
FROM orders;
```

Other patterns to recognize

Never did X: NOT EXISTS. **At least N related records:** GROUP BY + HAVING COUNT(...) >= N. **Above group average:** correlated subquery or AVG OVER (PARTITION BY ...). **Consecutive events:** LAG/LEAD or self-joins, with an explicit definition of consecutive. **Duplicate values:** GROUP BY key HAVING COUNT(*) > 1.

> Pattern recognition saves time only after you define the output grain, eligibility rules, denominator and tie behavior.

MY NOTES / QUESTIONS TO REVISIT

Worked business query

Find each region's top two customers by completed October revenue, including ties.

Break the request into decisions

Output grain: one winning customer per region. Eligibility: completed orders in October 2026. Revenue: sum of order totals. Top two: assume top two distinct revenue values because ties are included. Customers with no eligible orders are excluded. Rank only after customer totals are calculated.

```
WITH customer_revenue AS (
  SELECT c.region, c.customer_id,
         SUM(o.amount) AS revenue
  FROM customers AS c
  JOIN orders AS o
    ON o.customer_id = c.customer_id
  WHERE o.status = 'completed'
    AND o.order_ts >= '2026-10-01'
    AND o.order_ts <  '2026-11-01'
  GROUP BY c.region, c.customer_id
), ranked AS (
  SELECT region, customer_id, revenue,
         DENSE_RANK() OVER (
           PARTITION BY region ORDER BY revenue DESC
         ) AS revenue_rank
  FROM customer_revenue
)
SELECT region, customer_id, revenue
FROM ranked
WHERE revenue_rank <= 2
ORDER BY region, revenue DESC, customer_id;
```

A tiny manual test

East has customer totals 120, 120 and 90; all three qualify because the first two distinct totals are 120 and 90. A region with only one buyer returns that buyer. Orders at November 1 midnight are excluded; orders late on October 31 are included.

Defend the choices

INNER JOIN fits because only buyers are eligible. Grouping before ranking ensures we rank customer revenue, not individual orders. DENSE_RANK preserves revenue ties. A half-open timestamp range captures the full month. Final ORDER BY controls display order independently of the ranking window.

> If "top two" means exactly two customers, change to ROW_NUMBER and add a stable tie-breaker. Clarify ambiguity from the prompt or state a reasonable interpretation when the format permits.

MY NOTES / QUESTIONS TO REVISIT

Tables, constraints and schema design

Database questions may test what makes data valid, not just how to retrieve it.

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  region VARCHAR(40)
);
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  order_ts DATETIME NOT NULL,
  amount DECIMAL(12,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  CONSTRAINT fk_customer FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id),
  CONSTRAINT ck_amount CHECK (amount >= 0)
);
ALTER TABLE customers ADD COLUMN email VARCHAR(255);
CREATE VIEW completed_orders AS
SELECT order_id, customer_id, amount
FROM orders WHERE status = 'completed';
```

Keys and constraints

Primary key: uniquely identifies each row and is non-null; one primary-key constraint can contain multiple columns. **Candidate key:** any minimal unique identifier; one is chosen as primary. **Composite key:** multiple columns together identify a row. **Foreign key:** enforces a reference to a suitable key in another table, or the same table.

UNIQUE: prevents duplicate non-null key values; a nullable UNIQUE column in MySQL can contain multiple NULLs. **NOT NULL:** requires a value. **CHECK:** tests a condition; UNKNOWN can pass, so pair it with NOT NULL if missing values must be rejected. **DEFAULT:** supplies a value when a column is omitted (not generally when explicit NULL is supplied).

Relationships

A foreign key is not automatically unique; one parent may have many children. A many-to-many relationship usually uses a bridge table, such as enrollment(student_id, course_id), with a composite key. In InnoDB, ON DELETE CASCADE propagates parent deletion; RESTRICT and NO ACTION reject a conflicting delete; SET NULL requires nullable child columns.

CREATE / ALTER / DROP define objects. A view stores a query definition; an ordinary view does not guarantee stored results or ordering.

MY NOTES / QUESTIONS TO REVISIT

Data changes and transactions

Recognize DML syntax and how several changes become one logical unit.

```
INSERT INTO customers (customer_id, name, region)
VALUES (101, 'Asha', 'East'), (102, 'Ben', 'West');

UPDATE orders
SET status = 'completed'
WHERE order_id = 500;

DELETE FROM orders
WHERE order_id = 501;

START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
-- ROLLBACK cancels uncommitted transactional changes.
```

ACID

Atomicity: the transaction's changes succeed as a unit or are rolled back. **Consistency:** valid transactions preserve declared rules and application invariants. **Isolation:** controls how concurrent transactions interact. **Durability:** committed changes survive failures according to the database's guarantees.

A transfer also needs validation, such as sufficient funds and existence of both accounts, plus appropriate concurrency control. The short example illustrates syntax, not a complete transfer implementation. A SAVEPOINT allows a partial rollback within a transaction.

DELETE versus TRUNCATE versus DROP

DELETE: removes selected rows; supports WHERE. **TRUNCATE:** empties a table without row-level WHERE and may reset identity counters; in MySQL it causes an implicit commit and does not fire ON DELETE triggers. **DROP:** removes the table object itself.

> In MySQL, many DDL statements, including TRUNCATE, cause implicit commits and cannot be rolled back like ordinary transactional DML. Use InnoDB for the transaction examples in this workbook.

Without WHERE, UPDATE and DELETE target all rows. INSERT ... SELECT inserts query results. Parameterized queries keep user values separate from SQL structure and prevent injection through those values; dynamic identifiers require separate validation.

MY NOTES / QUESTIONS TO REVISIT

Indexes, plans and normalization

Know the tradeoffs well enough to answer a database concept question.

Indexes

An index is an auxiliary structure that can speed lookup, joins and ordering. B-tree-style indexes support many equality and range operations. They consume storage and add maintenance cost to inserts, updates and deletes. An index does not guarantee that the optimizer will use it.

```
CREATE INDEX idx_orders_customer_ts
ON orders (customer_id, order_ts);

EXPLAIN
SELECT order_id, amount
FROM orders
WHERE customer_id = 101
  AND order_ts >= '2026-10-01'
  AND order_ts <  '2026-11-01';
```

For a composite B-tree index (customer_id, order_ts), equality on the leading column plus a range on the next is a useful access pattern. A date-only query is generally less well served by this leading-column order; specialized optimizer strategies can be exceptions. Column order should follow the workload.

EXPLAIN: inspect access paths, join choices and estimated rows. MySQL EXPLAIN ANALYZE (8.0.18+) actually executes the query and reports timing and row counts. **Covering index:** an index contains the columns needed by the query, which may reduce base-table access. **Sargable predicate:** a condition that can support an index search, often a direct comparison/range on the indexed column.

Normalization

1NF: no repeating column groups; values are atomic for the chosen model. **2NF:** non-key attributes do not depend on only part of a candidate composite key. **3NF:** removes problematic transitive dependencies; formally, for each nontrivial dependency $X \rightarrow A$, X is a superkey or A is prime. **BCNF:** every determinant of a nontrivial dependency is a superkey.

Example: storing department_name on every employee repeats the dependency department_id → department_name and risks inconsistent updates. Put departments in a separate table and reference its key. Normalization reduces update, insert and delete anomalies. Deliberate denormalization trades some consistency complexity for read convenience/performance.

> Do not claim every index makes every query faster, every CTE is materialized, or every normalized design is automatically fastest.

MY NOTES / QUESTIONS TO REVISIT

MySQL isolation and syntax essentials

Use InnoDB transaction concepts and memorize the MySQL syntax you will actually write.

Isolation in InnoDB

InnoDB supports READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ and SERIALIZABLE. The default is **REPEATABLE READ**. Plain consistent reads in a repeatable-read transaction normally share the snapshot established by its first consistent read; at READ COMMITTED, each consistent read gets a fresh snapshot.

A plain snapshot SELECT and a locking read are different operations. SELECT ... FOR UPDATE locks rows for later modification within a transaction. Locking range reads can also involve gap/next-key locks. Deadlocks can occur; MySQL may roll back a victim transaction, which the application should retry.

Views and temporary results

An ordinary view stores a query definition; it does not store a refreshed result set. MySQL has no native materialized-view object. A CTE belongs to one statement. CREATE TEMPORARY TABLE creates a session-local table that can be referenced across statements until dropped or the session ends.

MySQL syntax essentials

```
-- Row limits and exact arithmetic
SELECT * FROM orders ORDER BY order_id LIMIT 5 OFFSET 10;
SELECT 7 / 2 AS ordinary_division, 7 DIV 2 AS integer_div;
-- Results: 3.5000 (display precision may vary), 3

-- Missing values: standard form and MySQL shorthand
SELECT COALESCE(NULL, 0), IFNULL(NULL, 0);

-- Identifiers use backticks when quoting is necessary.
SELECT `order_id` FROM `orders`;

-- Inspect a table definition or execution plan.
DESCRIBE orders;
SHOW CREATE TABLE orders;
EXPLAIN SELECT * FROM orders WHERE customer_id = 101;
```

> **Version checklist:** CTEs/windows: 8.0+. CHECK enforcement: 8.0.16+. EXPLAIN ANALYZE: 8.0.18+. INTERSECT/EXCEPT: 8.0.31+. MySQL has no native FULL OUTER JOIN and no QUALIFY clause.

Write string literals in single quotes. Prefer COALESCE and CASE for familiar, expressive solutions. MySQL IF(condition, yes_value, no_value) is available, but CASE handles multiple branches clearly.

MY NOTES / QUESTIONS TO REVISIT

Debugging and assessment checklist

Use a fixed review sequence instead of random query edits.

Before writing

1. Define the output grain and exact columns. **2.** Read key constraints and input types. **3.** Separate row eligibility from group eligibility. **4.** Define ties, missing data, denominator and date boundaries. **5.** Choose the smallest useful stages: filter → join → aggregate → rank → format.

When the answer is wrong

Too many rows: missing join condition, many-to-many fanout, wrong grouping columns, or tied rows that should be broken. **Too few:** inner join dropped unmatched records; right-table WHERE predicate defeated LEFT JOIN; NULL comparison removed candidates. **Inflated metric:** join multiplied fact rows or denominator uses wrong grain. **Empty result:** check NOT IN with NULL and date/type assumptions.

Hidden-test checklist

Test: empty input; one row; no matches; multiple matches; equal salaries/amounts; all values equal; missing dates; NULL values; zero denominator; repeated names; same month across two years; a timestamp exactly on the start/end boundary; duplicate business values with distinct IDs.

Submission check

Return exactly the requested columns and aliases. Add ordering only as required or for deterministic top-N selection. Check percentage versus proportion and decimal places. Avoid debugging SELECTs that produce extra result sets. Use MySQL functions and verify version support for newer set operations.

Practice thought process

A useful explanation: "Each row should be one customer. I preserve customers with zero orders using a left join, count the non-null order key, aggregate before ranking, and use dense rank because ties must survive." This explains intent, not merely syntax.

> For coding questions, the platform usually evaluates the result. You do not need to narrate every step unless requested. Use the thought process to structure your scratch work and SQL.

If stuck, inspect the intermediate result before aggregation. Count rows and distinct keys. Build a tiny counterexample by hand; it often reveals the incorrect assumption faster than rereading a large query.

MY NOTES / QUESTIONS TO REVISIT

Practice prompts - try before page 25

Use the shared schema on page 2. Write the grain and key edge case before the query.

A. Keep customers with zero purchases

Return customer_id and completed_order_count for every customer, including zero. Count only completed orders. Output ascending customer_id. Assume order_id is a non-null primary key.

B. Top two distinct salaries

Return employee_id, department_id and salary for employees earning one of the top two distinct non-null salaries in their department. Include every tied employee. Sort by department, salary descending and employee ID.

C. Completed order percentage

For each customer appearing in orders, return the percentage of their orders whose status is completed, rounded to two decimals. All orders count in the denominator, including ones with NULL status. Return customer_id and completed_pct.

D. Revenue grew from yesterday

Table daily_sales(day DATE PRIMARY KEY, revenue DECIMAL NOT NULL) may have missing dates. Return days whose revenue exceeds the revenue on the preceding calendar day. If the preceding day is absent, exclude the day. Sort ascending.

Explain without running SQL

E. A LEFT JOIN produces one unmatched customer row. What are COUNT(*) and COUNT(o.order_id) for that customer?
F. Salaries are 100, 100, 80, 60. What ranks do RANK and DENSE_RANK give?
G. Why can WHERE id NOT IN (SELECT nullable_id ...) remove every candidate?
H. Why can SUM(DISTINCT order_amount) be wrong after joining orders to items?

> Self-score each query: 1 point for correct grain, 1 for core syntax, 1 for NULL/tie/date handling, 1 for output requirements. Rework anything below 4/4 without looking at the answer.

MY NOTES / QUESTIONS TO REVISIT

Practice solutions and reasoning

Several equivalent solutions may be correct. These emphasize predictable edge-case behavior.

```
-- A. Right-side eligibility stays in ON.
SELECT c.customer_id,
       COUNT(o.order_id) AS completed_order_count
FROM customers AS c
LEFT JOIN orders AS o
  ON o.customer_id = c.customer_id
 AND o.status = 'completed'
GROUP BY c.customer_id
ORDER BY c.customer_id;

-- B. Rank distinct salary values, preserving ties.
WITH r AS (
  SELECT employee_id, department_id, salary,
         DENSE_RANK() OVER (
           PARTITION BY department_id ORDER BY salary DESC
         ) AS dr
  FROM employees WHERE salary IS NOT NULL
)
SELECT employee_id, department_id, salary
FROM r WHERE dr <= 2
ORDER BY department_id, salary DESC, employee_id;

-- C. ELSE 0 keeps unknown statuses in the denominator.
SELECT customer_id,
       ROUND(100.0 * SUM(CASE WHEN status = 'completed'
                              THEN 1 ELSE 0 END)
             / COUNT(*), 2) AS completed_pct
FROM orders
GROUP BY customer_id;

-- D. Compare actual calendar neighbors, not row neighbors.
SELECT a.day
FROM daily_sales AS a
JOIN daily_sales AS b
  ON a.day = DATE_ADD(b.day, INTERVAL 1 DAY)
WHERE a.revenue > b.revenue
ORDER BY a.day;
```

E: COUNT(*) = 1; COUNT(o.order_id) = 0. **F:** RANK = 1,1,3,4; DENSE_RANK = 1,1,2,3. **G:** with a NULL in the list, a candidate without an equal value can yield UNKNOWN, which WHERE rejects. **H:** equal amounts on different orders are legitimate and must not be collapsed. Aggregate at the order grain instead.

C has a nonzero denominator for each emitted group because every group comes from at least one order. An outer customer list would require a different denominator and explicit zero-order behavior.

MY NOTES / QUESTIONS TO REVISIT

One-page syntax reference

Recall the structure, then verify the exact metric and edge cases.

```
-- Retrieval skeleton
WITH stage AS (SELECT ... FROM ... WHERE ...)
SELECT group_key, SUM(value) AS total
FROM stage
JOIN other AS x ON stage.key = x.key
WHERE row_condition
GROUP BY group_key
HAVING SUM(value) > threshold
ORDER BY total DESC, group_key
LIMIT 10;

-- Conditional aggregate / safe rate
SUM(CASE WHEN condition THEN value ELSE 0 END)
COUNT(CASE WHEN condition THEN 1 END)
100.0 * numerator / NULLIF(denominator, 0)
COALESCE(expression, fallback)

-- Windows: filter their results in an outer query
ROW_NUMBER() OVER (PARTITION BY g ORDER BY t, id)
DENSE_RANK() OVER (PARTITION BY g ORDER BY score DESC)
LAG(value) OVER (PARTITION BY g ORDER BY t, id)
SUM(value) OVER (
  PARTITION BY g ORDER BY t, id
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

-- Exclusion without NOT IN's NULL trap
WHERE NOT EXISTS (
  SELECT 1 FROM child AS x WHERE x.parent_id = p.id
)

-- MySQL dates / strings / numbers
DATE(ts)                      DATE_FORMAT(ts, '%Y-%m')
DATE_ADD(d, INTERVAL 1 DAY)    DATEDIFF(end_d, start_d)
CONCAT(a,b)                   SUBSTRING(s,1,3)
TRIM(s)                       CHAR_LENGTH(s)
ROUND(x,2)                    CAST(x AS DECIMAL(12,2))

-- Mutations / definitions
INSERT INTO t (a,b) VALUES (1,'x');
UPDATE t SET b = 'y' WHERE a = 1;
DELETE FROM t WHERE a = 1;
CREATE INDEX idx_t_a ON t(a);
START TRANSACTION;  -- changes ... then COMMIT or ROLLBACK
```

> **Five checks:** right grain? correct join cardinality? correct denominator? NULL and ties handled? exact output and MySQL syntax?

MY NOTES / QUESTIONS TO REVISIT

Final review and reference notes

Use the remaining time to retrieve what you know, not just reread it.

A focused weekend review

First session: pages 3-11, then solve Customer Who Visited but Did Not Make Any Transactions, Managers with at Least 5 Direct Reports, Average Selling Price, Monthly Transactions I and Confirmation Rate.
Second session: pages 13-18, then solve Product Sales Analysis III, Second Highest Salary, Department Top Three Salaries and Restaurant Growth.
Before the assessment: pages 19-23 and 26; redo the practice prompts without looking. Use your own mistakes to select the final problems.

Official references

These links are optional follow-up references. The explanations and examples in this workbook are original study material; MySQL behavior should be checked against the version you use.

MySQL 8.4 manual: SELECT and related statement navigation
MySQL 8.4 manual: window function concepts and syntax
MySQL 8.4 manual: date and time functions
MySQL 8.4 manual: constraints and invalid data
MySQL 8.4 manual: InnoDB transaction isolation
LeetCode SQL 50 study plan

Personal review list

Three patterns I need to redo: _____

My most common mistake: _____

My MySQL version / weak topics: _____

> If you can explain grain, joins, grouping, NULLs, ties and date boundaries, you can reason through many unfamiliar intermediate questions.

MY NOTES / QUESTIONS TO REVISIT